

Plano de Execução Completo — Fase 1 TCC

Versão Revisada

Período: 17 de maio a 12 de junho

Equipe: P1, P2, P3

Convenção: [NOVO] = tarefa adicionada na revisão do cronograma (não constava na versão original)

SEMANA 1 — 17 a 23 de maio

P1 + P2 — Pipeline de Dados e Feature Engineering

P1 e P2 trabalham juntas do início ao fim da semana. Se uma travar, a outra ajuda imediatamente.

Segunda (17/05)

- Configurar ambiente conjunto — criar ambiente virtual e instalar bibliotecas:
 - pandas, numpy, scikit-learn, xgboost, lightgbm, fastf1, optuna, adapt, scipy, matplotlib, seaborn
 - Fixar versões em requirements.txt para garantir reprodutibilidade
- Conectar Ergast API e fazer primeira extração: resultados 2014–2024
 - Dividir período: P1 extrai 2014–2019, P2 extrai 2020–2024 — concatenar ao final
 - Campos: posições de grid, FinishPosition, status DNF, pontuação, pit stops
- Conectar FastF1: tempos de qualifying por setor e compostos de pneu por corrida (2014–2024)
 - Dividir o período entre P1 e P2 para popular o cache local mais rapidamente
- [NOVO] Conectar OpenF1 API e validar acesso
 - [NOVO] Instalar biblioteca ou configurar chamadas REST da OpenF1
 - [NOVO] Extrair uma corrida de 2025 (ex: GP da Austrália 2025) como teste de acesso
 - [NOVO] Verificar campos retornados: session_key, driver_number, position, date, grid_position
- [NOVO] Mapear campos OpenF1 → schema Ergast e documentar em mapeamento_openf1_ergast.md
 - [NOVO] Identificar campos presentes na OpenF1 mas ausentes no Ergast e vice-versa
 - [NOVO] Definir estratégia de fallback: quais campos de 2025/2026 precisam do Ergast como complemento
- Verificar completude geral: identificar corridas com dados ausentes em qualquer fonte
 - Documentar em dados_ausentes.txt para tratamento na quarta-feira

Terça (18/05)

- Limpeza do dataset histórico (Ergast + FastF1):
 - Remover registros com GridPosition ou FinishPosition nulos
 - Criar chave primária: RaceID = piloto + temporada + round

- Remover duplicatas via RaceID
- Filtrar apenas era híbrida 2014 em diante
- **[NOVO]** Definir e documentar tratamento de DNFs
 - **[NOVO]** Variante escolhida: DNF Excluded (alinhada com benchmark RAPM, MAE 2,3)
 - **[NOVO]** Separar DNFs de piloto (Collision, Accident, Spun-Off) vs carro (Engine, Gearbox, ERS...)
 - **[NOVO]** Registrar a decisão como escolha metodológica explícita na documentação
- Encoding:
 - One-Hot para circuito e construtor
 - Label Encoding ordinal para composto de pneu (Soft > Medium > Hard)
- Normalização:
 - Z-score para variáveis numéricas contínuas
 - MinMaxScaler para GridPosition e Laps

Quarta (19/05)

- Tratamento de valores ausentes:
 - Tempos de volta: mediana do circuito naquele ano
 - Composto de pneu: moda da corrida
 - Qualifying (0,07% ausente): imputação KNN
- Tratamento de outliers:
 - Critério: valores acima de 3 desvios padrão da média por circuito
 - Outliers legítimos (safety car, falha mecânica): manter com flag `safety_car_flag = 1`
 - Outliers espúrios (erro de sensor, dado corrompido): remover
- Dataset limpo entregue — base para feature engineering

Quinta (20/05)

- **[NOVO]** Implementar `rapm_ridge.py` — modelo auxiliar de coeficientes RAPM
 - **[NOVO]** Construir matriz esparsa binária de indicadores piloto e construtor por corrida
 - **[NOVO]** Aplicar Ridge Regression com time-decay sobre toda a base histórica 2014–2024
 - **[NOVO]** Iterar corrida a corrida (para cada corrida r , treinar em todas as anteriores a r)
 - **[NOVO]** Gerar `coef_pilotos.csv` e `coef_construtores.csv` com coeficientes históricos
 - **[NOVO]** Opcional: suavizar coeficientes via LOESS conforme descrito no RAPM paper
- Feature engineering parte 1 (usar coeficientes RAPM gerados acima):
 - `driver_coef_rapm`: coeficiente de habilidade do piloto via Ridge time-decay
 - `constructor_coef_rapm`: coeficiente de competitividade do construtor via Ridge time-decay
 - `recent_form_5`: média ponderada das últimas 5 corridas (pesos 5,4,3,2,1)
 - `recent_form_3`: média ponderada das últimas 3 corridas (pesos 3,2,1)

Sexta (21/05)

- Feature engineering parte 2:
 - `driver_experience`: total de corridas na carreira até aquela data
 - `driver_wins_total`: total de vitórias na carreira até aquela data
 - `driver_dnf_rate`: DNFs do piloto / corridas disputadas
 - `constructor_dnf_rate`: DNFs mecânicos / corridas do construtor
 - `constructor_wins_total`: total de vitórias do construtor até aquela data
 - `driver_constructor_synergy`: média histórica do piloto com essa equipe específica

Fim de semana (22–23/05)

- Feature engineering parte 3:
 - `track_complexity`: score normalizado de comprimento + curvas + elevação + incidentes históricos
 - `weather_impact_factor`: score normalizado de temperatura + umidade + precipitação
 - `avg_pit_stops_circuit`: média histórica de pit stops por circuito
- Integrar todas as features ao dataset limpo
- Tratar NaN gerados pelo feature engineering (ex: primeiras corridas de piloto sem histórico)
- Análise de correlação:
 - Matriz de correlação entre todas as features
 - Identificar pares com $r > 0,85$
 - Documentar decisões de remoção
- **[NOVO]** Extrair temporada 2025 completa via OpenF1
 - **[NOVO]** Com mapeamento validado na segunda, extrair todas as corridas de 2025 disponíveis
 - **[NOVO]** Aplicar mesmo pipeline de limpeza do histórico (RaceID, DNF, encoding, normalização)
 - **[NOVO]** Salvar em `openf1_2025_clean.csv` — será o conjunto de validação walk-forward da S2
- **[NOVO]** Enviar mapeamento `openf1_ergast.md` para P3 até sábado 23/05
 - **[NOVO]** P3 precisará desse documento na segunda seguinte para escrever a subseção de fontes

Features que entram no modelo — lista final após correlação

Categoria	Feature	Descrição
Grid	<code>grid_position</code>	Posição de largada após qualifying
Forma recente	<code>recent_form_5</code>	Média ponderada últimas 5 corridas
Forma recente	<code>recent_form_3</code>	Média ponderada últimas 3 corridas
Piloto	<code>driver_experience</code>	Total de corridas na carreira
Piloto	<code>driver_wins_total</code>	Total de vitórias na carreira
Piloto	<code>driver_coef_rapm</code>	Coefficiente de habilidade via Ridge time-decay
Piloto	<code>driver_dnf_rate</code>	Taxa histórica de DNF do piloto
Construtor	<code>constructor_coef_rapm</code>	Coefficiente de competitividade via Ridge time-decay
Construtor	<code>constructor_dnf_rate</code>	Taxa histórica de DNF mecânico
Construtor	<code>constructor_wins_total</code>	Total de vitórias do construtor
Sinergia	<code>driver_constructor_synergy</code>	Desempenho médio do piloto com essa equipe
Circuito	<code>circuit_type</code>	Urbano vs permanente (binária)
Circuito	<code>track_complexity</code>	Score combinado de complexidade
Circuito	<code>altitude</code>	Altitude do circuito em metros
Pneu	<code>tire_compound_start</code>	Composto de largada

Pneu	avg_pit_stops_circuit	Média histórica de pit stops por circuito
Temporada	season_factor	Ano da corrida — captura evolução tecnológica
Clima	weather_impact_factor	Score combinado de condições climáticas
Flag	safety_car_flag	Corrida teve safety car historicamente

Features excluídas — documentar na metodologia

Feature	Motivo
finish_position	Dado pós-corrida — data leakage
race_points	Calculado do resultado — data leakage
fastest_lap_race	Dado pós-corrida — data leakage
previous_position	Captura padrão trivial que quebra em 2026 — exclusão consciente e documentada

Entregável interno — 23/05

- P1+P2: dataset_limpo.csv
- P1+P2: dataset_features.csv
- P1+P2: pipeline_dados.py
- P1+P2: feature_engineering.py
- **[NOVO]** P1+P2: rapm_ridge.py
- **[NOVO]** P1+P2: coef_pilotos.csv
- **[NOVO]** P1+P2: coef_construtores.csv
- **[NOVO]** P1+P2: openf1_test_au2025.json
- **[NOVO]** P1+P2: mapeamento_openf1_ergast.md
- **[NOVO]** P1+P2: openf1_2025_clean.csv
- P1+P2: relatorio_correlacao.pdf
- P3: introducao.docx
- P3: revisao_literatura.docx

P3 — Introdução + Revisão da Literatura

Segunda (17/05)

- Rascunho completo da Introdução (2–3 páginas):
 - Contextualização da F1 como problema de ML
 - O problema da transição regulatória de 2026
 - Pergunta de pesquisa do TCC
 - Estrutura do documento

Terça (18/05)

- Revisão da Literatura — subseção 1: Predição de resultados em F1
 - Cobrir todos os artigos revisados
 - Apresentar distinção Grupo 1 vs Grupo 2
 - Explicar por que acurácias de 99% não são benchmarks válidos

Quarta (19/05)

- Revisão da Literatura — subseção 2: Transfer Learning
 - Pan & Yang (2010) — definições formais, tipos, negative transfer
 - Zhuang et al. (2021) — survey completo, abordagens modernas
 - Conectar com o problema do TCC: transfer homogêneo, transdutivo para indutivo

Quinta (20/05)

- Revisão da Literatura — subseção 3: Concept Drift
 - Lu et al. (2019) — tipos de drift, detecção, adaptação
 - MUSCATEL — pesos temporais decrescentes
 - Handling Concept Drift GFM — dois modelos adaptativos
 - NBA LSTM paper — impacto de mudanças em modelos esportivos

Sexta (21/05)

- Revisão da Literatura — subseção 4: Lacunas da literatura
 - Nenhum trabalho mede velocidade de recuperação do MAE durante transição regulatória
 - RAPM menciona o problema mas não o resolve
 - Bayesian paper evita o problema limitando o período
 - Essa subseção justifica a existência do TCC — escrever com cuidado

Fim de semana (22–23/05)

- Formatar tudo em ABNT: fontes, margens, espaçamento, citações
- Revisão completa da Introdução e Revisão da Literatura
- Preparar estrutura vazia das seções seguintes para preencher nas próximas semanas
- **[NOVO]** Receber mapeamento_openf1_ergast.md de P1/P2 e estudar para escrever subseção de fontes na S2

SEMANA 2 — 24 a 30 de maio

P1 + P2 — Modelagem

Segunda (24/05)

- Implementar walk-forward validation:
 - Treino 2014–2022 → Validação 2023
 - Treino 2014–2023 → Validação 2024
 - Treino 2014–2024 → Validação 2025 (dados: openf1_2025_clean.csv)
 - Treino 2014–2025 → Drift test 2026 (executado na S3, não aqui — ver segunda 31/05)
- **[NOVO]** Integrar openf1_2025_clean.csv como conjunto de validação 2025
 - **[NOVO]** Garantir que o pipeline de features roda corretamente sobre o openf1_2025_clean.csv
 - **[NOVO]** Verificar compatibilidade de schema antes de rodar os modelos
- **[NOVO]** Otimizar fator de time-decay via grid-search
 - **[NOVO]** Testar fatores: 0,50 / 0,65 / 0,75 / 0,85 / 0,95
 - **[NOVO]** Avaliar via walk-forward em 2023–2024, selecionar fator que minimiza MAE
 - **[NOVO]** Documentar valor escolhido e justificativa (o fator 0,75 do RAPM é ponto de partida, não fixo)
- **[NOVO]** Implementar metrics.py com todas as métricas customizadas
 - **[NOVO]** Kendall τ : calcular correlação de rankings por corrida e fazer média entre corridas
 - **[NOVO]** Acurácia top-3: função customizada que verifica se o podium previsto bate com o real
 - **[NOVO]** Wrapper geral: função que calcula MAE, RMSE, R^2 , Kendall τ e acurácia top-3 de uma vez
- Implementar time-decay com fator otimizado:
 - Calcular peso de cada corrida e integrar ao sample_weight do XGBoost e Random Forest

Terça (25/05)

- Rodar XGBoost sem tuning — verificar que o pipeline funciona do início ao fim
- Rodar Random Forest sem tuning — verificar que o pipeline funciona
- Calcular métricas preliminares com metrics.py: MAE, RMSE, R^2 , Kendall τ , acurácia top-3
- Identificar e corrigir eventuais problemas no pipeline

Quarta (26/05)

- Tuning Optuna — XGBoost, 50 trials:

Parâmetro	Faixa
n_estimators	100–500
max_depth	3–10
learning_rate	0,01–0,3
subsample	0,6–1,0
reg_alpha (L1)	0–1
reg_lambda (L2)	0–1

Quinta (27/05)

- Tuning Optuna — Random Forest, 50 trials:

Parâmetro	Faixa
n_estimators	100–500
max_depth	3–15
max_features	sqrt, log2, 0.5
min_samples_split	2–10
min_samples_leaf	1–5

Sexta (28/05)

- **[NOVO]** Rodar Ridge Regression baseline com tuning de lambda via RidgeCV
 - **[NOVO]** Varrer alpha: faixa 0,01 a 100 em escala log com cross-validation temporal
 - **[NOVO]** Registrar o alpha ótimo — baseline forte é essencial para validar ganho dos modelos de árvore
- Calcular métricas finais preliminares dos 3 modelos com hiperparâmetros otimizados:
 - MAE, RMSE, R^2 , Kendall τ , acurácia top-3
 - **[NOVO]** Reportar MAE $\pm$ desvio padrão calculado sobre os folds do walk-forward
- Montar tabela comparativa completa

Fim de semana (29–30/05)

- RFE com XGBoost: ranking de importância das features
- Testar remoção das features menos importantes: verificar se MAE piora
- Definir conjunto final de 12–15 features
- **[NOVO]** Re-rodar modelos com conjunto reduzido e recalculando métricas finais definitivas
 - **[NOVO]** As métricas finais devem refletir o conjunto de features definitivo, não o completo
- Feature importance comparativa: XGBoost vs Random Forest
- **[NOVO]** Salvar explicitamente feature_importance_2024.csv do fold 2024
 - **[NOVO]** Esse arquivo será comparado com a importância em 2026 na análise de drift da S3
- Verificar se os dois algoritmos valorizam as mesmas features
- **[NOVO]** Verificar cobertura de 2025 na OpenF1 e documentar lacunas
 - **[NOVO]** Quantas corridas de 2025 estão disponíveis? Alguma com dados incompletos?
 - **[NOVO]** Se houver corrida faltando: definir se usa Ergast como fallback e documentar a decisão
- **[NOVO]** Alinhar equipe sobre cronograma preliminar da Fase 2 (julho–novembro)
 - **[NOVO]** P3 precisará de um esboço desse cronograma para escrever a seção de Próximos Passos na S3

Entregável interno — 30/05

- P1+P2: walk_forward.py
- **[NOVO]** P1+P2: metricas.py
- P1+P2: tuning_xgboost.py
- P1+P2: tuning_randomforest.py

- **[NOVO]** P1+P2: otimizacao_time_decay.py
- **[NOVO]** P1+P2: otimizacao_ridge_lambda.py
- P1+P2: tabela_metricas.csv (XGBoost vs RF vs Ridge — com desvio padrão)
- P1+P2: feature_importance_xgb.csv
- P1+P2: feature_importance_rf.csv
- **[NOVO]** P1+P2: feature_importance_2024.csv
- P3: metodologia.docx (rascunho completo)

P3 — Metodologia

Segunda (24/05)

- Metodologia — subseção: Fonte e coleta dos dados
 - Ergast API, FastF1, OpenF1 — usar mapeamento_openf1_ergast.md recebido de P1/P2
 - Período de análise e justificativa do corte em 2014
 - Volume de dados: corridas, pilotos, circuitos

Terça (25/05)

- Metodologia — subseção: Tratamento dos dados
 - Limpeza, valores ausentes, outliers, encoding, normalização
 - Decisão sobre variante de DNF (Excluded) com justificativa bibliográfica
 - Cada decisão com justificativa bibliográfica

Quarta (26/05)

- Metodologia — subseção: Feature engineering
 - Descrever cada feature: fórmula e justificativa
 - Descrever o modelo auxiliar RAPM Ridge para geração dos coeficientes de piloto e construtor
 - Análise de correlação e decisões de remoção
 - Justificar a não aplicação do PCA global

Quinta (27/05)

- Metodologia — subseção: Modelagem
 - Descrever XGBoost e Random Forest com diferenças filosóficas
 - Walk-forward validation e time-decay — incluir a otimização do fator de decay
 - Ridge Regression baseline com RidgeCV — não tratar como modelo sem tuning
 - Optuna para hiperparâmetros
 - Métricas de avaliação e metas baseadas na literatura — incluir formulação do Kendall τ

Sexta (28/05)

- Metodologia — subseção: Posicionamento metodológico
 - Por que o TCC é Grupo 2 e não Grupo 1
 - Quais features foram excluídas e por quê
 - Como o trabalho se diferencia dos estudos existentes

Fim de semana (29–30/05)

- Revisão completa da Metodologia
- Integrar com Introdução e Revisão da Literatura
- Verificar consistência entre as seções

SEMANA 3 — 31 de maio a 5 de junho

As três trabalham em paralelo — o código está pronto e cada uma tem responsabilidade independente.

P1 — Documentação do Código

Segunda (31/05)

- **[NOVO]** Criar script de atualização incremental `update_openf1_2026.py`
 - **[NOVO]** Consultar a OpenF1 e verificar quais corridas de 2026 já foram baixadas
 - **[NOVO]** Acrescentar apenas corridas novas ao CSV existente — sem re-baixar tudo
 - **[NOVO]** Esse script será reutilizado na Fase 2 ao longo de toda a temporada de 2026
- Documentar `pipeline_dados.py`:
 - Docstring em cada função
 - Comentários explicando cada decisão de limpeza
 - Justificativa bibliográfica nos comentários críticos

Terça (01/06)

- Documentar `feature_engineering.py`:
 - Fórmula de cada feature nos comentários
 - Referência ao artigo que originou cada feature
- **[NOVO]** Documentar `rapm_ridge.py`:
 - **[NOVO]** Descrever a lógica da Ridge iterativa e o time-decay nos comentários
 - **[NOVO]** Comentar a opção de suavização LOESS e quando ativá-la

Quarta (02/06)

- Criar notebook Jupyter de demonstração:
 - Pipeline completo rodando do início ao fim
 - Resultados reais (não placeholder)
 - Visualizações inline
 - Seed fixo para reprodutibilidade

Quinta (03/06)

- Criar README completo:
 - Instruções de instalação do ambiente
 - Como reproduzir os resultados do zero
 - Descrição de cada arquivo do repositório
 - **[NOVO]** Incluir seção sobre `update_openf1_2026.py` e como rodar para novas corridas

Sexta (04–05/06)

- Revisão cruzada: ler seção de Resultados do P3 e verificar se os números batem com o código
- Ajustes finais na documentação

P2 — Análises Finais e Visualizações

Segunda (31/05)

- **[NOVO]** Extrair corridas de 2026 disponíveis via OpenF1
 - **[NOVO]** Rodar `update_openf1_2026.py` para baixar todas as corridas de 2026 já realizadas
 - **[NOVO]** Aplicar pipeline de limpeza e features — salvar em `openf1_2026_available.csv`
- Aplicar modelo treinado em 2014–2025 nas corridas de 2026 (drift test)
 - **[NOVO]** Protocolo: aplicar o modelo a cada corrida de 2026 individualmente
 - **[NOVO]** Calcular MAE corrida a corrida — não agregado, mas ponto a ponto na série temporal
- Construir curva de degradação: MAE por corrida de 2025 entrando em 2026

Terça (01/06)

- Análise de erro por tipo de circuito:
 - MAE em circuitos urbanos vs permanentes
 - XGBoost vs RF — qual performa melhor em cada tipo
- Análise de erro por posição no grid:
 - O modelo é mais preciso para os primeiros do grid?
 - Erro para pilotos do meio vs fundo do grid

Quarta (02/06)

- Feature importance antes e depois da transição regulatória:
 - Usar `feature_importance_2024.csv` (salvo na S2) como referência do 'antes'
 - Calcular feature importance nas primeiras corridas de 2026 como referência do 'depois'
 - Comparar: quais features mudam mais de importância entre as duas eras
 - Esperado: `constructor_coef_rapm` muda drasticamente em 2026

Quinta (03/06)

- Comparar degradação XGBoost vs Random Forest em 2026:
 - XGBoost degrada mais rápido ou mais devagar?
 - Qual tem maior variância nas primeiras corridas da nova era?
 - Documentar a diferença de comportamento
- **[NOVO]** Análise contrafactual
 - **[NOVO]** Projetar o MAE esperado em 2026 caso não houvesse drift (extrapolando tendência de 2025)
 - **[NOVO]** Sobrepor à curva real de MAE em 2026 — a diferença entre as duas é o impacto regulatório
 - **[NOVO]** Essa análise isola o efeito da mudança regulatória de outros fatores (novos pilotos, circuitos)
- **[NOVO]** Atualizar `openf1_2026_available.csv` antes de gerar gráficos finais
 - **[NOVO]** Rodar `update_openf1_2026.py` para incluir corridas realizadas entre segunda e quinta

Sexta (04–05/06)

- Finalizar todas as visualizações em qualidade de publicação:
 - Curva de MAE ao longo de 2025

- Curva de drift entrando em 2026 com linha contrafactual sobreposta
- Feature importance comparativa XGBoost vs RF (2024 vs primeiras corridas de 2026)
- Tabela de métricas final com desvio padrão entre folds
- Cada gráfico com título, eixos, legenda e fonte

P3 — Resultados Parciais + Próximos Passos + Integração

Segunda (31/05)

- Resultados — subseção: Comparativo de modelos
 - Tabela XGBoost vs Random Forest vs Ridge Regression (com desvio padrão entre folds)
 - Discussão: resultados consistentes com a literatura?
 - Comparar MAE obtido com TabNet (2,17) e RAPM (2,3)

Terça (01/06)

- Resultados — subseção: Feature importance
 - Quais variáveis dominam em cada algoritmo
 - XGBoost e Random Forest valorizam as mesmas features?
 - Confirmar se `grid_position` e `constructor_coef_rapm` dominam

Quarta (02/06)

- Resultados — subseção: Quantificação do drift em 2026
 - Curva de MAE de 2025 entrando em 2026
 - Quanto o modelo degrada nas primeiras corridas
 - XGBoost degrada diferente do Random Forest?
 - Análise contrafactual: magnitude do impacto regulatório isolado

Quinta (03/06)

- Próximos Passos (1–2 páginas):
 - Apresentar arquitetura da Fase 2: TrAdaBoost
 - Justificar com literatura de concept drift
 - Definir métrica de velocidade de recuperação do MAE
 - Cronograma julho–novembro — usar esboço alinhado com a equipe no fim de semana 29–30/05

Sexta (04–05/06)

- Integrar todos os documentos em arquivo único ABNT:
 - Capa, sumário, introdução, revisão da literatura, metodologia, resultados parciais, próximos passos, referências
 - Numeração de páginas, figuras e tabelas
 - Todas as citações no texto têm entrada nas referências
 - Revisão final de formatação

Entregável interno — 05/06

- P1: `codigo_fase1/` completo e documentado
- P1: `notebook_demonstracao.ipynb`

- P1: README.md
- **[NOVO]** P1: update_openf1_2026.py
- P2: graficos/ todas as visualizações em qualidade final
- P2: analyse_drift_2026.csv
- P2: analyse_por_circuito.csv
- **[NOVO]** P2: openf1_2026_available.csv
- **[NOVO]** P2: analyse_contrafactual.csv
- P3: tccl_fasel_completo.docx versão quase final

SEMANA DE FOLGA — 6 a 12 de junho

Revisão cruzada obrigatória: ninguém entrega sem que outra pessoa tenha lido. Inconsistências entre código e texto são o erro mais comum em TCCs.

Métricas de Sucesso da Fase 1

Métrica	Meta	Referência
MAE (posições)	$\leq 2,5$	TabNet: 2,17 / RAPM: 2,3
RMSE	$\leq 3,0$	TabNet: 2,87
R^2	$\geq 0,75$	TabNet: 0,75
Kendall τ	$\geq 0,60$	RAPM: 0,625
Acurácia top-3	$\geq 70\%$	Polishchuk: 78%

Pontos de Atenção

Dependência crítica Semana 1:

P1 e P2 trabalham juntas do início ao fim. O mapeamento da OpenF1 na segunda-feira é pré-requisito para a extração de 2025 no fim de semana e para a validação do walk-forward na semana seguinte. Se a OpenF1 apresentar problema de schema ou acesso, parar tudo e resolver antes de avançar.

Coefficientes RAPM são um pipeline separado:

O `rapm_ridge.py` não é uma feature calculada em uma linha — é um modelo auxiliar completo com matriz esparsa, Ridge iterativa e time-decay. Reservar a quinta inteira da S1 para isso. Os coeficientes `driver_coef_rapm` e `constructor_coef_rapm` só existem depois que esse script rodar.

Fator de time-decay deve ser otimizado, não fixado:

O valor 0,75 vem do RAPM paper e é um ponto de partida. O grid-search na segunda da S2 pode encontrar um fator diferente para este dataset específico. Usar 0,75 sem verificar é uma decisão metodológica indefensável na banca.

Dados de 2026:

Até 12 de junho haverá entre 8 e 10 corridas disponíveis. Suficiente para mostrar a curva de drift inicial e a análise contrafactual. A recuperação completa via TrAdaBoost é o objetivo da Fase 2.

Se o MAE ficar acima de 2,5:

Não é catástrofe. Documentar honestamente, comparar com os benchmarks do Grupo 2 e usar como motivação adicional para a Fase 2. Um MAE de 3,0 ainda está dentro do estado da arte real de previsão genuína em F1.

Revisão cruzada obrigatória:

Ninguém entrega sem que outra pessoa tenha lido. Inconsistências entre código e texto são o erro mais comum em TCCs e a semana de folga existe para isso.